

Part 1: Tree Fundamentals

Binary Trees and Tree Data Structures

Why trees exist

Arrays, vectors and linked lists all store data in a straight line. That works when the data itself is a line: marks of 50 students, characters of a string, prices over 30 days.

A lot of real data is not a line. It is a hierarchy.

```
Company
|-- Engineering
|   |-- Backend
|   +-- Frontend
+-- Sales
    |-- India
    +-- Europe
```

Folders inside folders, an HTML page inside body inside html, a tournament bracket, a family lineage, the decision steps of a game. In all of these, one item leads to several items below it, and each of those again leads to more. You cannot express that with a single line of memory without a lot of pain.

There is a second reason, a purely practical one. A sorted array searches in $O(\log N)$ but inserting in the middle costs $O(N)$. A linked list inserts in $O(1)$ but searches in $O(N)$. A balanced search tree gives you both search and insert in $O(\log N)$. Databases, file systems, `std::map` and `std::set` are all built on trees for exactly this reason.

So trees exist for two reasons:

1. Some data is naturally hierarchical.
2. Some operations are only fast when data is arranged in a branching structure.

For this course we care mostly about the first reason, because that is what LeetCode tree problems test.

What is a tree

A tree is a set of **nodes** connected by **edges**, with three rules:

1. One node is marked special. It is called the **root**, and it is drawn at the top.
2. Every other node has exactly one node directly above it (its parent).

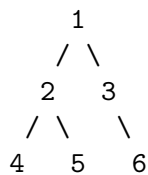
3. There are no cycles. You can never walk down and come back to where you started.

Because of rule 2 and 3, a tree with N nodes always has exactly $N - 1$ edges. Every node except the root consumes exactly one edge to attach itself to its parent.

Computer science trees are drawn upside down. Root on top, leaves at the bottom. Get used to it now, every diagram in this material follows that convention.

The tree we will use everywhere

Here is the tree that will appear again and again in this material. Memorise its shape. When a new algorithm is introduced you should be able to predict its output on this tree before reading the answer.



Node 1 is the root. Nodes 4, 5 and 6 have no children. Node 3 has only a right child.

Terminology

Everything below refers to the tree above.

Node: a single element. It holds a value and links to its children.

Edge: the link between a parent and a child. The pair (2, 4) is an edge.

Root: the topmost node, node 1. It has no parent.

Parent: node 2 is the parent of 4 and 5.

Child: 4 and 5 are children of 2. In a binary tree we say left child and right child, and the difference matters.

Siblings: nodes with the same parent. 4 and 5 are siblings. 2 and 3 are siblings.

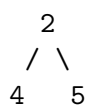
Leaf: a node with no children. Here: 4, 5, 6. A leaf is a node where both `left` and `right` are `NULL`.

Internal node: any node that is not a leaf. Here: 1, 2, 3.

Ancestor: any node on the path from a node up to the root. Ancestors of 5 are 2 and 1.

Descendant: everything below a node. Descendants of 2 are 4 and 5.

Subtree: a node together with all of its descendants. The subtree rooted at 2 is



This word is the most important one on the page. A subtree is itself a complete, valid tree. That single fact is what makes recursion work on trees.

Level: nodes at the same distance from the root form a level. Level 0 = {1}, level 1 = {2, 3}, level 2 = {4, 5, 6}.

Depth of a node: number of edges from the root down to that node. $\text{Depth}(1) = 0$, $\text{depth}(2) = 1$, $\text{depth}(4) = 2$. Depth is measured looking upward, toward the root.

Height of a node: number of edges on the longest path from that node down to a leaf. $\text{Height}(4) = 0$, $\text{height}(2) = 1$, $\text{height}(1) = 2$. Height is measured looking downward.

Height of the tree: the height of its root. Here, 2.

Depth versus height, and the counting trap

Depth and height are measured in opposite directions. A leaf has height 0. The root has depth 0.

There is a second trap. Some books count **edges**, some count **nodes**. A single node alone has height 0 by the edge definition and height 1 by the node definition. Both are used in the real world.

LeetCode uses the node count. Problem 104, “Maximum Depth of Binary Tree”, says the depth of the tree above is 3, not 2.

Rule for this course:

- When we write a LeetCode style function called `maxDepth` or `height`, we count **nodes**, so an empty tree returns 0 and a single node returns 1.
- When we discuss theory such as “the height of a perfect tree is $\log N$ ”, we count **edges**.

Whenever you sit down to code, decide which one you are using in the first ten seconds, and make the base case agree with it. Almost every off-by-one bug in tree code comes from skipping this step.

Binary tree

A **binary tree** is a tree where every node has at most two children, and the two children are distinguishable: one is the left child, one is the right child.

“Distinguishable” is not a detail. These two trees are different trees:



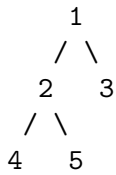
The first has 2 as a left child, the second as a right child. Their inorder traversals differ, their views differ, and a function like “is this tree symmetric” will give different answers.

Almost every interview and contest tree problem is about binary trees, which is why we spend the whole course on them.

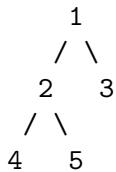
Special shapes of binary trees

You need these names to read problem statements. You do not need formal proofs.

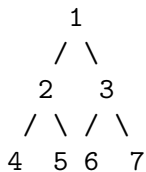
Full binary tree: every node has either 0 or 2 children. No node has exactly one child.



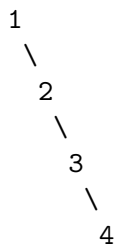
Complete binary tree: every level is completely filled except possibly the last, and the last level is filled from left to right with no gaps. This is the shape a heap uses, and it is the shape that lets you store a tree in a plain array.



Perfect binary tree: every internal node has 2 children and all leaves are on the same level.



Skewed or degenerate tree: every node has only one child. It is a linked list wearing a tree costume.



Why the shape matters

Two numbers you should be able to state instantly:

- A perfect tree of height h has $2^{(h+1)} - 1$ nodes. Level d holds at most 2^d nodes.
- Therefore a balanced tree with N nodes has height about $\log_2(N)$. For $N = 10^6$ that is about 20.

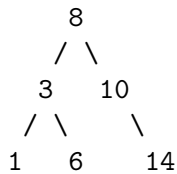
A skewed tree with N nodes has height $N - 1$.

Every recursive tree algorithm uses $O(H)$ stack space, where H is the height. On a balanced tree that is 20 stack frames. On a skewed tree of 10^5 nodes that is 10^5 stack frames, and on some judges that is a stack overflow. When you see “the tree can be skewed” in constraints, that is the hint.

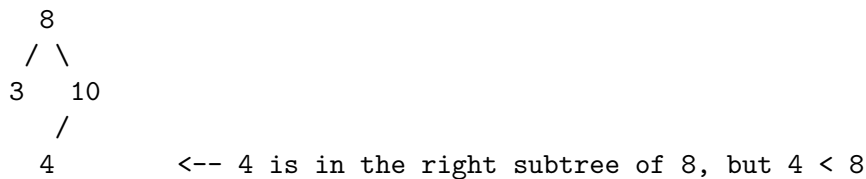
Binary Search Tree versus Binary Tree

A **Binary Search Tree (BST)** is a binary tree with one extra promise:

For every node, every value in its left subtree is smaller than the node, and every value in its right subtree is larger.



Note the word **every**. It is not enough that the immediate left child is smaller. The whole left subtree must be smaller. This tree is **not** a BST even though each node looks locally fine:



The single most useful consequence: **the inorder traversal of a BST is sorted**. We will prove that visually in Part 2.

For the rest of the course, assume a tree is a plain binary tree with arbitrary values unless the problem explicitly says BST. Students lose marks and interview points by assuming order that was never given.

Representing a tree in C++

This is the structure used by LeetCode and by everything in this material:

```

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;

    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};
  
```

Three fields. A value, a pointer to the left child, a pointer to the right child. NULL means “there is no child there”.

Two observations that carry the whole course:

1. **A tree is just a pointer to its root.** There is no separate Tree object. When a function takes `TreeNode* root`, it has received an entire tree.
2. **Every node is the root of its own subtree.** So `root->left` is not “a child”, it is “a smaller tree”. Passing `root->left` into the same function is completely legal and completely meaningful.

That is the whole idea behind tree recursion. Nothing more.

Building our tree by hand

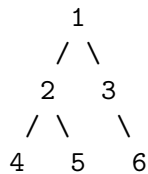
```

TreeNode* buildSampleTree() {
    TreeNode* root = new TreeNode(1);
  
```

```

root->left = new TreeNode(2);
root->right = new TreeNode(3);
root->left->left = new TreeNode(4);
root->left->right = new TreeNode(5);
root->right->right = new TreeNode(6);
return root;
}

```



Keep this function in your practice file. When you want to test any algorithm from this material locally, build this tree and run on it.

The array representation

A complete binary tree can be stored in a plain array with no pointers at all:

- node at index i has children at $2*i + 1$ and $2*i + 2$
- its parent is at $(i - 1) / 2$

```

index:  0  1  2  3  4  5  6
value:  1  2  3  4  5  6  7

```

This is how heaps and segment trees are built. For general binary trees it wastes memory (a skewed tree of 20 nodes would need an array of size 2^{20}), so LeetCode problems use pointers.

How LeetCode writes trees in the input

You will see input like:

```
[1,2,3,4,5,null,6]
```

That is level order, left to right, with `null` marking a missing child. Read it as:

```

level 0:  1
level 1:  2 3
level 2:  4 5 null 6

```

which is exactly our tree. You never have to parse this yourself in the editor, but you must be able to draw the tree from it on paper. Practice that until it is instant.

Why recursion fits trees so naturally

A quick recursion refresher

A recursive function calls itself on a smaller version of the same problem. It needs two parts:

1. a **base case**, small enough to answer directly
2. a **recursive step**, which solves a smaller input and builds the answer from it

```
int fact(int n) {
    if (n == 0) return 1;           // base case
    return n * fact(n - 1);        // smaller version of the same problem
}
```

When you call `fact(3)`, the machine does not magically know the answer. It stacks up pending work:

```
fact(3) waits for fact(2)
    fact(2) waits for fact(1)
        fact(1) waits for fact(0)
            fact(0) = 1                <- base case, now unwind
        fact(1) = 1 * 1 = 1
    fact(2) = 2 * 1 = 2
fact(3) = 3 * 2 = 6
```

Two things to notice, because they reappear in every tree problem:

- The going-down phase and the coming-back-up phase are different moments. You can do work in either one.
- Every pending call sits on the **call stack**, which costs memory.

Now the tree version

For an array you had to invent the smaller problem: $n - 1$, or “the rest of the array”. In a tree the smaller problem is already sitting there. `root->left` is a smaller tree. `root->right` is a smaller tree. The structure hands you the recursion for free.

So every tree function looks like this:

```
ReturnType solve(TreeNode* root) {
    if (root == NULL) return answerForEmptyTree;

    ReturnType L = solve(root->left);    // answer for the left subtree
    ReturnType R = solve(root->right);   // answer for the right subtree

    return combine(L, R, root->val);     // build my answer from theirs
}
```

Read that template again. Roughly seventy percent of binary tree problems on LeetCode are this template with a different `combine` and a different `ReturnType`. Part 5 and Part 6 are entirely about filling in those two blanks.

The base case is almost always `root == NULL`. Ask yourself: what is the correct answer for an empty tree? Number of nodes: 0. Height: 0. Sum: 0. Maximum value: negative infinity. Is it balanced: yes. That question, asked honestly, gives you the base case every single time.

What the calls look like on our tree

```
solve(1)
|-- solve(2)
|   |-- solve(4)
|   |   |-- solve(NULL)
|   |   +-- solve(NULL)
```

```

|    +-- solve(5)
|        |-- solve(NULL)
|        +-- solve(NULL)
+-- solve(3)
    |-- solve(NULL)
    +-- solve(6)
        |-- solve(NULL)
        +-- solve(NULL)

```

Every node is entered exactly once, so the work is $O(N)$ as long as `combine` is $O(1)$. The deepest chain of pending calls is the height of the tree, so the extra space is $O(H)$: $O(\log N)$ for a balanced tree, $O(N)$ in the worst case.

Learn to draw this diagram. When a recursive tree solution misbehaves, drawing the call tree for a 5 node example finds the bug faster than any debugger.

Common mistakes

- **Forgetting the NULL base case.** Then `root->val` dereferences a null pointer and you get a runtime error or a segfault. Write the `if (root == NULL)` line first, before anything else.
- **Checking the child instead of the node.** Writing `if (root->left == NULL) return ...` at the top means an empty tree still crashes, and the logic gets duplicated for both sides. Guard the node itself.
- **Mixing up height and depth.** Depth counts downward from the root, height counts upward from the leaves.
- **Mixing up edge counting and node counting.** Decide first, then write the base case to match.
- **Assuming a plain binary tree is a BST.** Only assume ordering when the problem says so.
- **Assuming a leaf means “one child is NULL”.** A leaf means **both** children are NULL. This one specific mistake breaks the Minimum Depth problem for almost everyone the first time.
- **Forgetting that the tree can be a single long chain.** Recursion depth equals height, not $\log N$.

Check yourself

1. Our sample tree has 6 nodes. How many edges does it have, and why can you answer without looking at the picture?
2. What is the height of node 3, and what is the depth of node 3?
3. A binary tree has height 4 counting edges. What is the largest number of nodes it can contain? What is the smallest?
4. Why is a subtree also a tree, and why does that matter for writing code?
5. For a function that counts the number of nodes, what should the answer be when `root == NULL`, and how did you decide?
6. Draw the tree for the LeetCode input `[3,9,20,null,null,15,7]`.